Climora

Application web météo en microservices



Équipe : Mariem Aissa · Ala Dhouib · Yosr Saad
Malek Dabbek · Hanin Lamouchi · Joumena Turki

Sommaire

1 *Introduction*

2 *Problématique*

3 *Fonctionnalités*

4 *Acteurs*

5 *Diagramme de cas d'utilisation*

6 *Diagramme de classe d’Analyse*

7 *Architecture technique globale*

8 *Découpage en microservices*

9 *Diagramme de communication*

Introduction

La météo influence nos décisions quotidiennes, de nos déplacements à nos activités.

Pourtant, les informations météorologiques restent souvent dispersées entre plusieurs plateformes : conditions actuelles, prévisions, alertes et suivi des villes.

Notre projet vise à centraliser ces informations au sein d'une seule application web, tout en offrant des fonctionnalités personnalisées et un espace participatif permettant aux utilisateurs de partager et de vérifier les mises à jour météorologiques.

Problématique



Comment concevoir une application web centralisée permettant de gérer les villes, consulter les données météorologiques, suivre les prévisions et recevoir des alertes et recommandations personnalisées ?

Fonctionnalités générales

Consultation des conditions météorologiques en temps réel

Consultation des prévisions météorologiques sur 7 jours.

Réception d'alertes et de notifications personnalisées.

Recommandation d'activités selon les conditions météo

Suivi météo de plusieurs villes

Partage et vérification communautaire

Acteurs



Visiteur

Consulte la météo, les prévisions et les signalements ; s'inscrit et se connecte.



Utilisateur

Rôle USER : favoris, alertes, notifications, activités, signalements et votes.



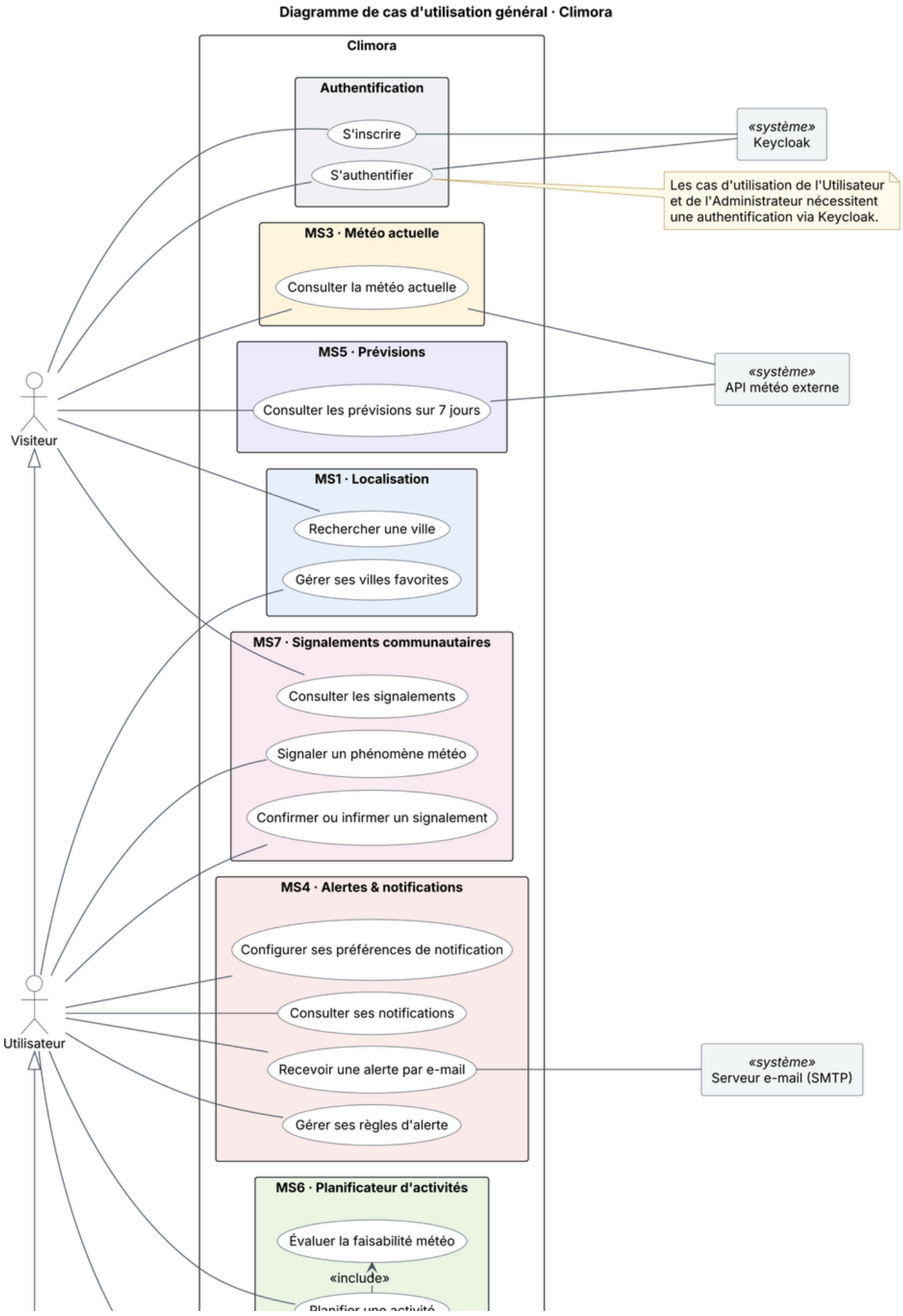
Administrateur

Rôle ADMIN : gère les référentiels et modère les signalements.



Systèmes externes

Keycloak, API météo et serveur e-mail (SMTP).


Diagramme de cas d'utilisation général · Climora
Climora
Authentification
S'inscrire
S'authentifier
«système»
Keycloak
Les cas d'utilisation de l'Utilisateur et de l'Administrateur nécessitent une authentification via Keycloak.
MS3 · Météo actuelle
Consulter la météo actuelle
MS5 · Prévisions
Consulter les prévisions sur 7 jours
«système»
API météo externe
Visiteur
MS1 · Localisation
Rechercher une ville
Gérer ses villes favorites
MS7 · Signalements communautaires
Consulter les signalements
Signaler un phénomène météo
Confirmer ou infirmer un signalement
MS4 · Alertes & notifications
Configurer ses préférences de notification
Consulter ses notifications
Recevoir une alerte par e-mail
Gérer ses règles d'alerte
«système»
Serveur e-mail (SMTP)
Utilisateur
MS6 · Planificateur d'activités
Évaluer la faisabilité météo
«include»
Planifier une activité

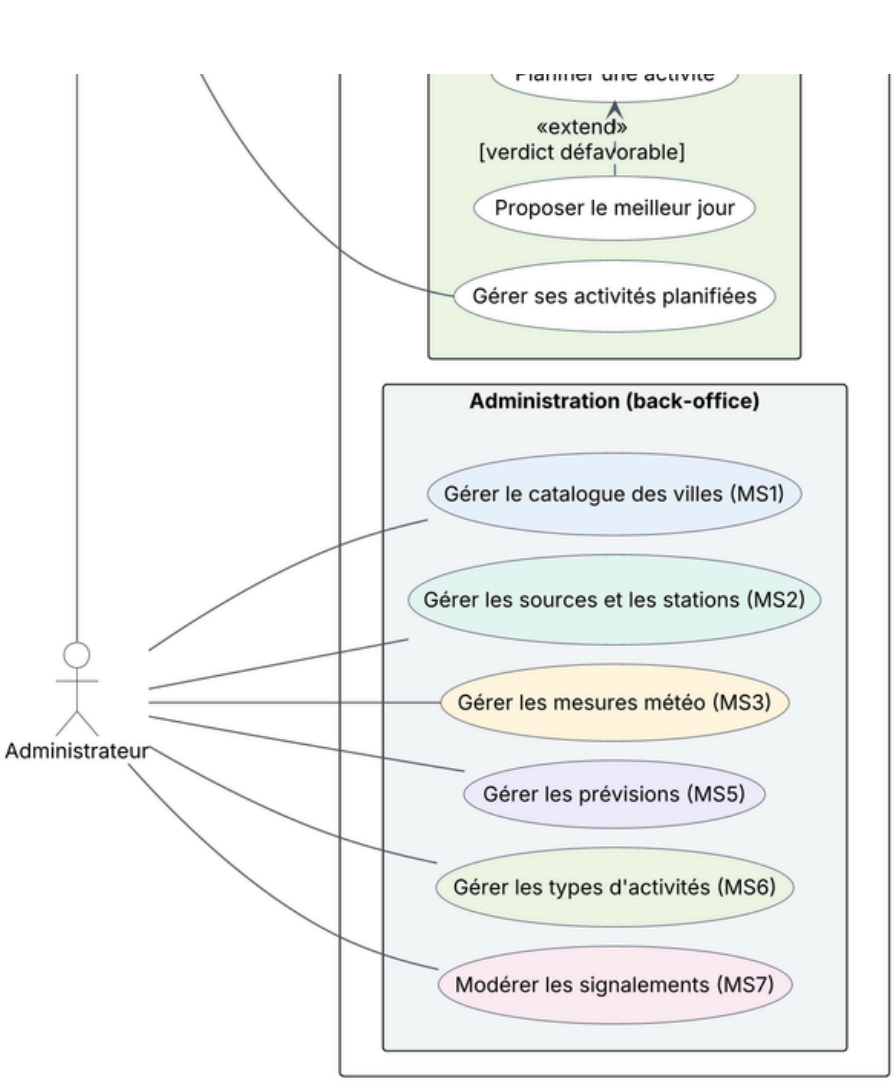
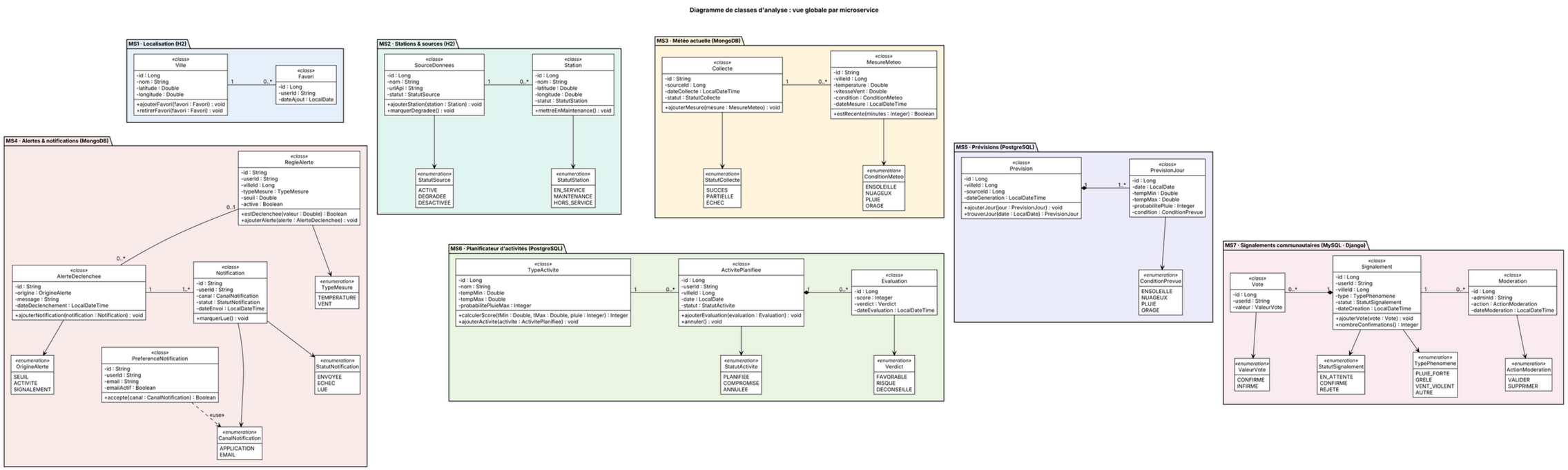
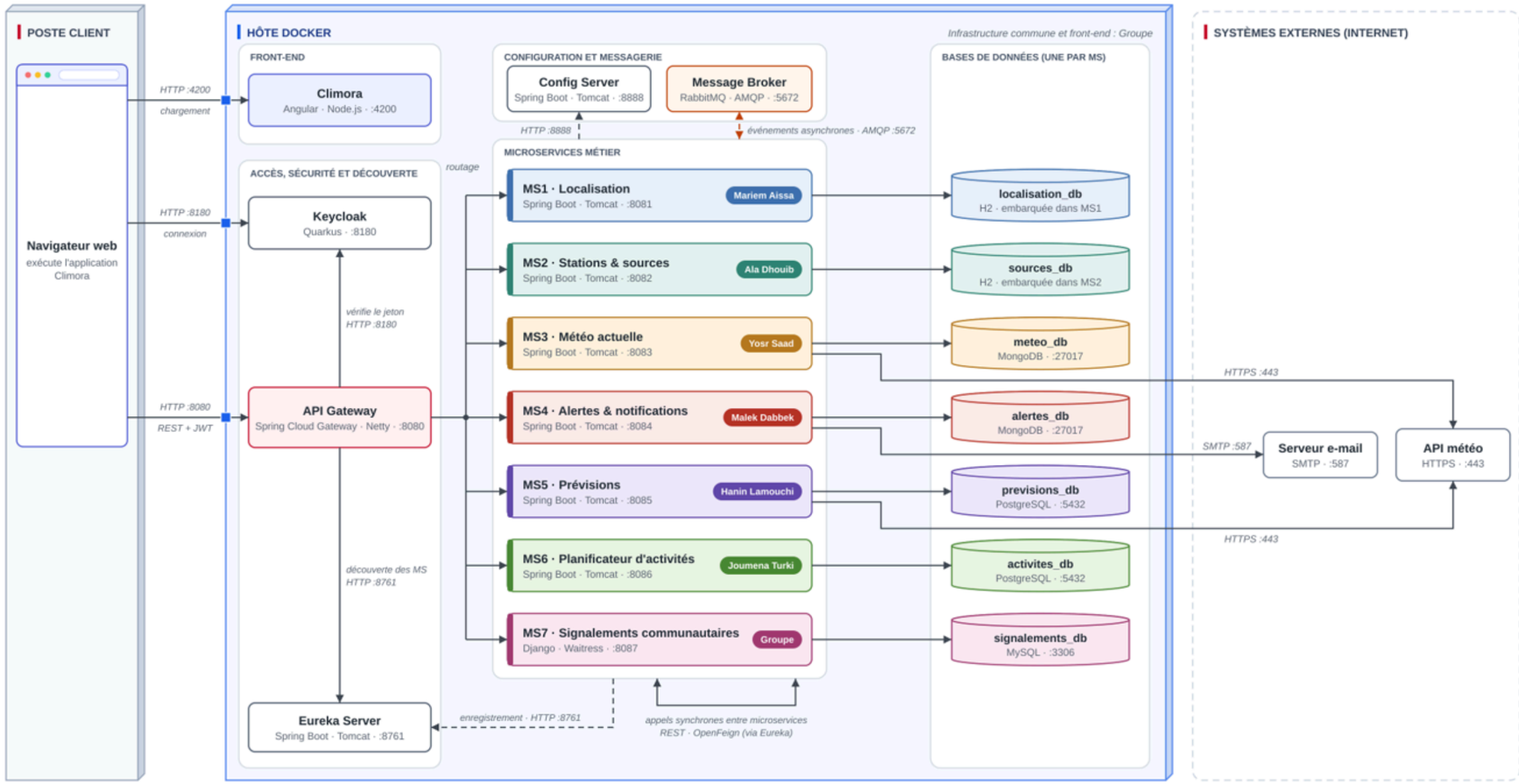

«extend»
[verdict défavorable]
Proposer le meilleur jour
Gérer ses activités planifiées
Administration (back-office)
Gérer le catalogue des villes (MS1)
Gérer les sources et les stations (MS2)
Gérer les mesures météo (MS3)
Gérer les prévisions (MS5)
Gérer les types d'activités (MS6)
Modérer les signalements (MS7)
Administrateur

Diagramme de classes



Architecture technique



Découpage en microservices

Sept microservices, chacun avec sa propre base de données.



Localisation

MS1 · Mariem Aissa



Stations & sources

MS2 · Ala Dhouib



Météo actuelle

MS3 · Yosr Saad



Alertes & notifications

MS4 · Malek Dabbek



Prévisions

MS5 · Hanin Lamouchi



Planificateur d'activités

MS6 · Joumena Turki



Signalements communautaires

MS7 · Groupe (Django)

MS1 : Localisation

Responsabilité

Gère le catalogue des villes (nom, coordonnées) et les villes favorites des utilisateurs. Entités : Ville et Favori (1 ville, 0 à plusieurs favoris)

Base : H2 (embarquée)

Données de référence peu volumineuses, modèle relationnel simple, aucun serveur à installer. Aucune clé étrangère vers un autre service : seulement userId (Keycloak).

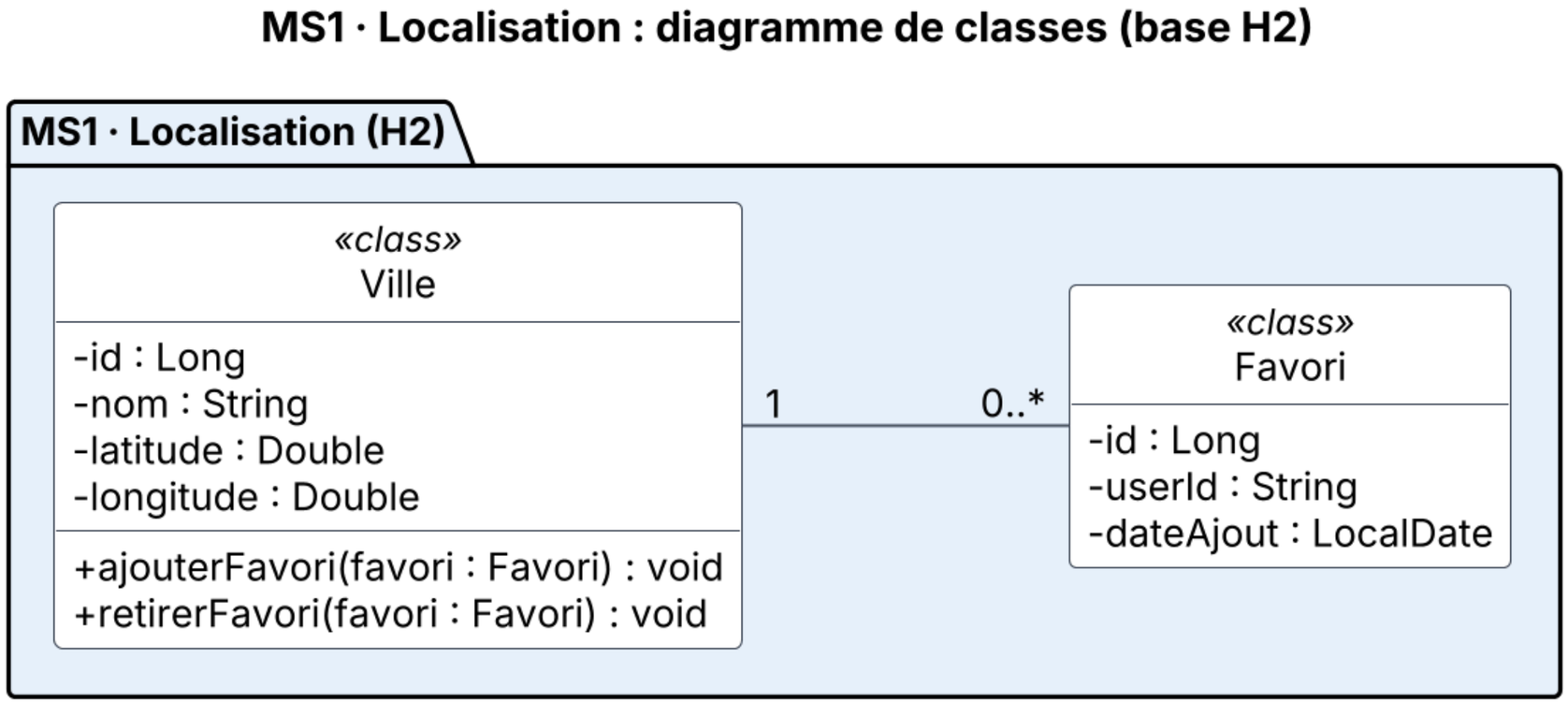
Méthode	Endpoint	Description	Codes	Accès
GET	`/api/villes?nom={nom}`	Rechercher une ville	200	Public
GET	`/api/villes/{id}`	Détail d'une ville (appelé par MS5 et MS7)	200, 404	Public
GET	`/api/villes/suivies`	Villes ayant au moins un favori (appelé par MS3)	200	Interne
POST	`/api/villes`	Ajouter une ville au catalogue	201, 400, 409	ADMIN
PUT	`/api/villes/{id}`	Modifier une ville	200, 400, 404	ADMIN
DELETE	`/api/villes/{id}`	Supprimer une ville	204, 404	ADMIN
GET	`/api/favoris`	Lister ses villes favorites	200	USER
POST	`/api/favoris`	Ajouter une ville favorite	201, 404, 409	USER
DELETE	`/api/favoris/{id}`	Retirer une ville favorite	204, 404	USER

Lien	Mode	Données échangées	Pourquoi	Si indisponible
MS3 → MS1	Synchrone, OpenFeign	Reçoit la liste des villes suivies	La collecte a besoin de la liste avant de commencer	MS3 garde ses dernières données
MS5 → MS1	Synchrone, OpenFeign	MS5 envoie villeId, reçoit latitude et longitude	L'API de prévisions est interrogée par coordonnées	Dernières prévisions en base, sinon 503
MS7 → MS1	Synchrone, OpenFeign (client HTTP Python)	MS7 envoie villeId, reçoit la ville ou 404	Vérifier que la ville du signalement existe	Signalement refusé

Sécurité

Authentification : JWT avec Keycloak.
Autorisation : Contrôle d'accès basé sur les rôles USER et ADMIN.
ADMIN : Gestion complète des villes.
USER : Gestion personnelle des favoris.

Diagramme de classes Localisation





MS2 ·Stations & sources

Responsabilité

Gère le catalogue des sources de données météo et des stations de mesure, et indique la source active. Entités : SourceDonnees et Station (1 source, 0 à plusieurs stations).

Base : H2

Petit catalogue administré, rarement modifié, avec une relation simple entre sources et stations. Base embarquée dans le microservice. Aucune clé étrangère vers un autre service : MS2 ne garde aucun identifiant externe

Méthode	Endpoint	Rôle	Succès	Erreurs	Accès
GET	/api/sources	Lister les sources	200	401, 403	ADMIN
GET	/api/sources/active	Source active et ses stations (appelé par MS3 et MS5)	200	404	Interne
POST	/api/sources	Ajouter une source	201	400, 401, 403	ADMIN
PUT	/api/sources/{id}	Modifier une source (statut)	200	400, 401, 403, 404	ADMIN
POST	/api/sources/{id}/stations	Ajouter une station à une source	201	400, 401, 403, 404	ADMIN
PATCH	/api/stations/{id}/maintenance	Mettre une station en maintenance	200	401, 403, 404	ADMIN
DELETE	/api/stations/{id}	Supprimer une station	204	401, 403, 404	ADMIN

MS2 ·Stations & sources

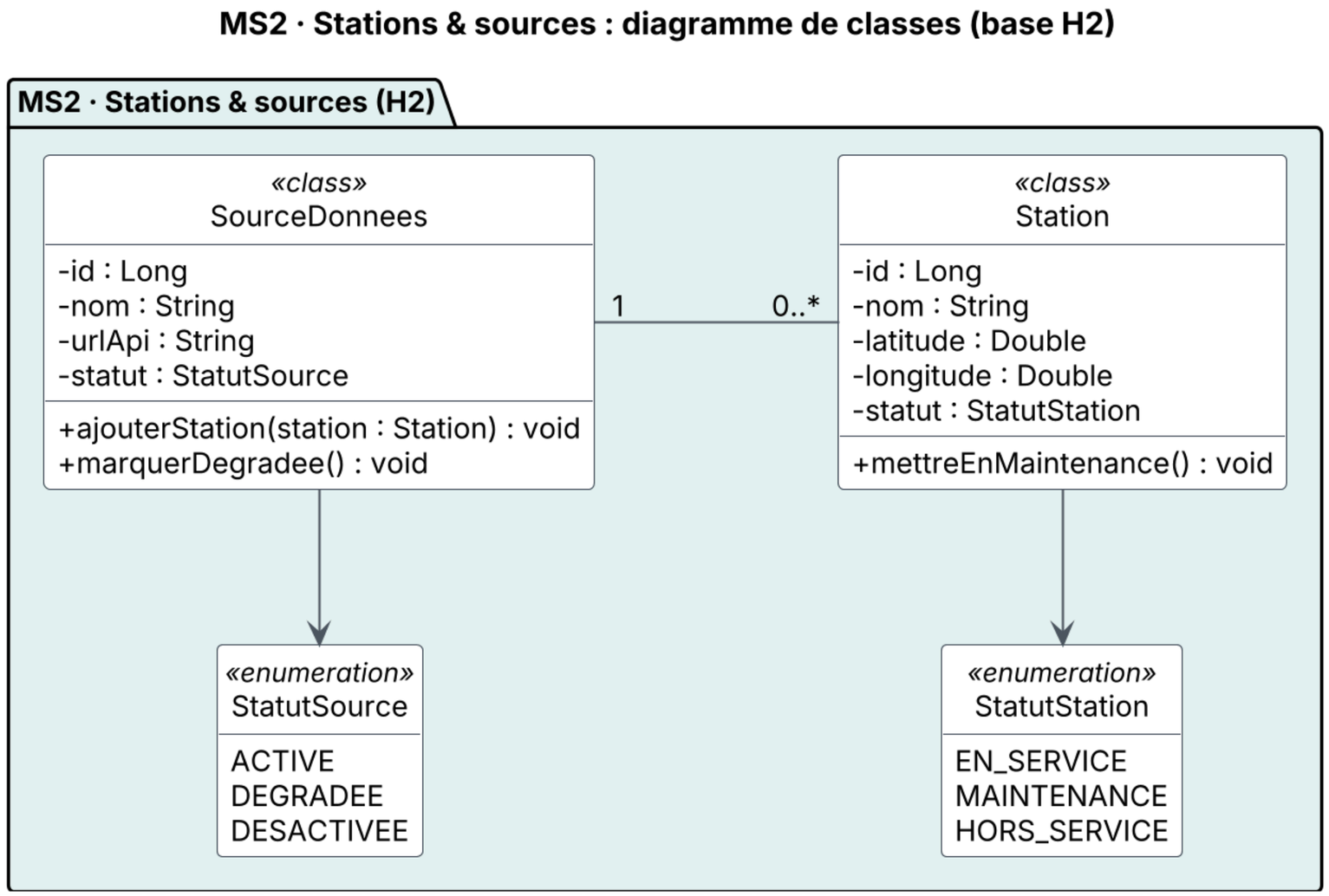
Appelés : aucun

Appelant : MS3, MS5

Evénement aucun

Lien	Mode	Données échangées	Pourquoi	Si indisponible
MS3 → MS2	Synchrone, OpenFeign	Reçoit la source active (URL de l'API) et ses stations	Savoir quelle API interroger pour la collecte	Dernières mesures en base
MS5 → MS2	Synchrone, OpenFeign	Reçoit la source active (URL de l'API)	Savoir quelle API interroger	Dernières prévisions en base, sinon 503

Diagramme de classe Stations & Sources



MS3 · Météo actuelle

Responsabilité

Collecte la météo actuelle des villes suivies depuis la source active, stocke les mesures et les expose aux autres services. Entités : Collecte et MesureMeteo (1 collecte, 0 à plusieurs mesures).

Base : MongoDB

Mesures nombreuses, horodatées, écrites souvent et lues par ville, sans jointure. Aucune clé étrangère vers un autre service : seulement villeId et sourceId.

Contrat d’API

Méthode	Endpoint	Rôle	Succès	Erreurs	Accès
GET	/api/meteo/villes/{villeId}	Météo actuelle d’une ville (appelé par MS6 et MS7)	200	404	Public
GET	/api/collectes	Lister les collectes	200	401, 403	ADMIN
POST	/api/collectes	Lancer une collecte manuelle	202	401, 403, 503	ADMIN
GET	/api/mesures	Lister les mesures	200	401, 403	ADMIN
PUT	/api/mesures/{id}	Corriger une mesure	200	400, 401, 403, 404	ADMIN
DELETE	/api/mesures/{id}	Supprimer une mesure	204	401, 403, 404	ADMIN



MS3 · Météo actuelle : Interactions

Appelés : **MS1, MS2, API météo externe**

Appelants : **MS6, MS7**

Événement vers : **MS4 (RabbitMQ)**

Lien	Mode	Données échangées	Pourquoi	Si indisponible
MS3 → MS1	Synchrone, OpenFeign	Reçoit la liste des villes suivies	La collecte a besoin de la liste avant de commencer	MS3 garde ses dernières données
MS3 → MS2	Synchrone, OpenFeign	Reçoit la source active (URL de l'API) et ses stations	Savoir quelle API interroger pour la collecte	Dernières mesures en base
MS3 → API météo	HTTPS	Envoie les coordonnées, reçoit la météo actuelle	Source des données	Mesures déjà stockées
MS6 → MS3	Synchrone, OpenFeign	MS6 envoie villeId, reçoit la météo actuelle	Mesure réelle pour le jour même	MS6 gère l'erreur (repli sur MS5, sinon 503)
MS7 → MS3	Synchrone, OpenFeign (client HTTP Python)	MS7 envoie villeId, reçoit la mesure officielle	Comparer le signalement à la mesure officielle	MS7 gère l'erreur (404, 503)
MS3 → MS4	Asynchrone, RabbitMQ	Publie WeatherUpdated (villeId, température, vent, date)	Aucune réponse attendue, services découplés	Événement gardé dans la file, MS3 non bloqué

MS3 · Météo actuelle : Sécurité

Authentification

JWT émis par Keycloak, vérifié par l’API Gateway puis par MS3.

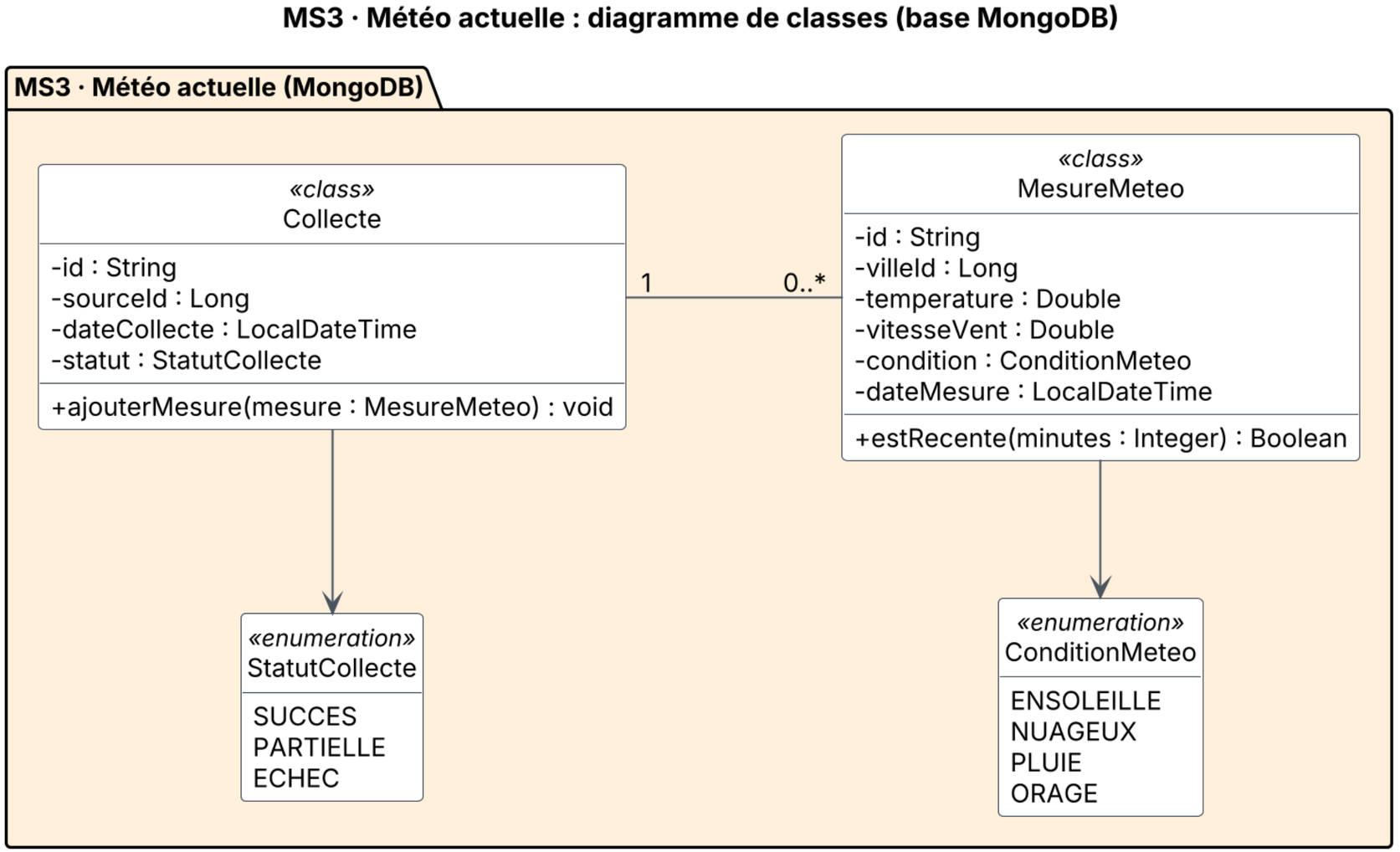
La consultation de la météo actuelle reste publique : un visiteur n’a pas besoin de compte.

Autorisation

Contrôle d’accès basé sur les rôles.

Public : consulter la météo actuelle d’une ville.
ADMIN : gérer les collectes et les mesures (consulter, lancer, corriger, supprimer).
Appels internes (MS6, MS7) : via OpenFeign, adresses résolues par Eureka(retrouver l'adresse d'un autre grâce à son nom logique.).

Diagramme de classe Météo Actuelle





MS4· Alertes & notifications

Responsabilité

Gère les règles d'alerte des utilisateurs, déclenche les alertes et envoie les notifications (application, e-mail).
Entités : RegleAlerte, AlerteDeclenchee, Notification, PreferenceNotification.
Ne gère pas : météo (MS3), villes (MS1), prévisions (MS5), authentification (Keycloak)

Base : MongoDB

Alertes et notifications nombreuses, au contenu variable selon l'origine, consultées par utilisateur, sans jointure: **MongoDB convient**. Aucune clé étrangère vers un autre service : seulement userId et villeId.

Événements consommés

WeatherUpdated,
ForecastUpdated,
ActivityRiskDetected,
NewWeatherReport

Méthode	Endpoint	Rôle	Succès	Erreurs	Accès
GET	/api/regles	Lister ses règles	200	401	USER
POST	/api/regles	Créer une règle	201	400, 401	USER
PUT	/api/regles/{id}	Modifier ou désactiver	200	400, 401, 404	USER
DELETE	/api/regles/{id}	Supprimer une règle	204	401, 404	USER
GET	/api/notifications	Consulter ses notifications	200	401	USER
PATCH	/api/notifications/{id}/lue	Marquer comme lue	200	401, 404	USER
GET	/api/preferences	Lire ses préférences	200	401	USER
PUT	/api/preferences	Modifier ses préférences	200	400, 401	USER



MS4· Alertes & notifications: Interactions

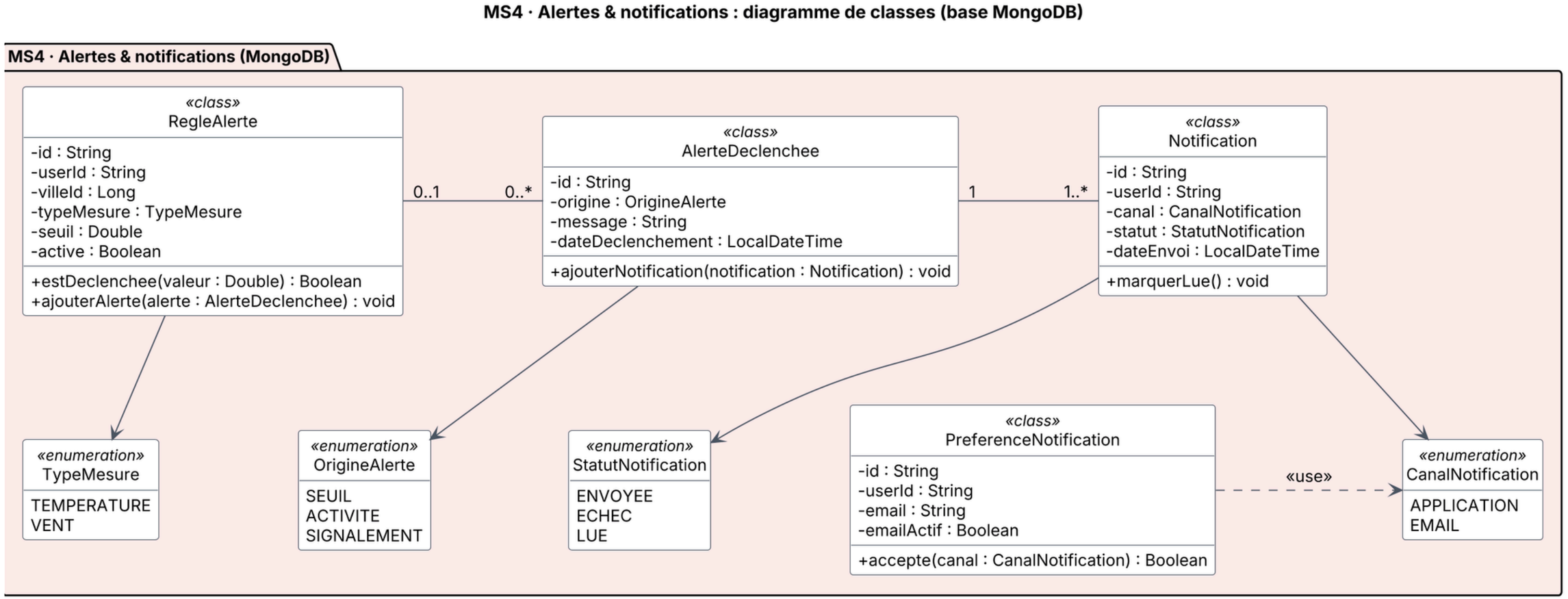
Événements reçus de MS3, MS5, MS6, MS7

Appelant : front-end via la Gateway

Appelé : serveur e-mail (SMTP)

Lien	Mode	Données échangées	Pourquoi	Si indisponible
MS3 → MS4	Asynchrone, RabbitMQ	WeatherUpdated : nouvelle mesure enregistrée (ville, température, vent, date)	Aucune réponse attendue, services découplés	RabbitMQ conserve l'événement dans une file jusqu'à son traitement ; la collecte de MS3 n'est pas bloquée
MS5 → MS4	Asynchrone, RabbitMQ	ForecastUpdated : prévisions mises à jour (ville)	Aucune réponse attendue, services découplés	RabbitMQ conserve l'événement dans une file jusqu'à son traitement ; les prévisions de MS5 ne sont pas bloquées
MS6 → MS4	Asynchrone, RabbitMQ	ActivityRiskDetected : activité devenue risquée (utilisateur concerné)	Aucune réponse attendue, services découplés	RabbitMQ conserve l'événement dans une file jusqu'à son traitement ; MS6 n'est pas bloqué
MS7 → MS4	Asynchrone, RabbitMQ	NewWeatherReport : nouveau signalement (ville)	Aucune réponse attendue, services découplés	RabbitMQ conserve l'événement dans une file jusqu'à son traitement ; MS7 n'est pas bloqué
MS4 → serveur SMTP	Synchrone, SMTP	E-mail d'alerte (destinataire, message)	Savoir si l'envoi a réussi (statut ENVOYEE ou ECHEC)	Notification e-mail en statut ECHEC ; la notification dans l'application est conservée
Front → MS4	Synchrone, REST via la Gateway	Règles, notifications, préférences (JSON) avec le jeton JWT	L'utilisateur attend la réponse	La requête échoue ; MS4 garde ses données et les événements en attente sont traités à son retour

Diagramme de classe Alertes & Notifications



MS5 · Prévisions

Responsabilité

Fournit les prévisions sur 7 jours par ville, actualisées depuis la source météo active. Entités : Prevision et PrevisionJour (1 prévision, 1 à 7 jours).

Base : PostgreSQL

Les données de MS5 sont liées entre elles.
 PostgreSQL, base de données relationnelle, répond naturellement à ce besoin : il organise ces données en tables reliées et garantit leur cohérence

Méthode	Endpoint	Rôle	Succès	Erreurs	Accès
GET	/api/previsions/villes/{villeId}	7 jours d'une ville	200	404	Public
GET	/api/previsions/villes/{villeId}/jours/{date}	Un jour (appelé par MS6)	200	400, 404	Public
POST	/api/previsions/actualisations	Forcer l'actualisation	202	401, 403, 503	ADMIN
DELETE	/api/previsions/{id}	Supprimer une prévision	204	401, 403, 404	ADMIN

MS5 · Prévisions: Interactions

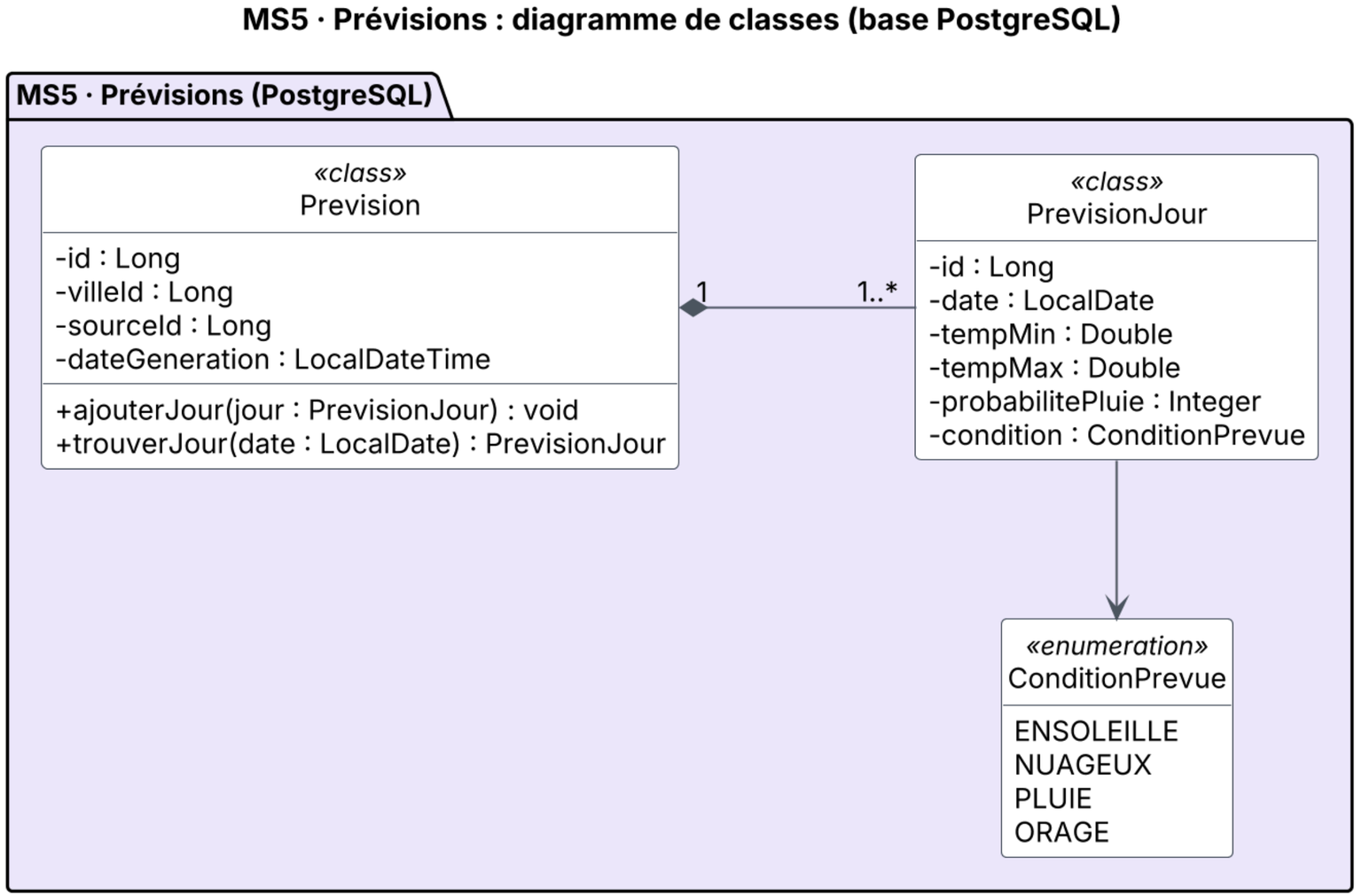
Appelés : MS1, MS2, API météo externe

Appelant : MS6

Evénement vers :MS4

Lien	Mode	Données échangées	Pourquoi	Si indisponible
MS5 → MS1	Synchrone, OpenFeign	Reçoit latitude et longitude de la ville	Coordonnées nécessaires tout de suite	Dernières prévisions en base, sinon 503
MS5 → MS2	Synchrone, OpenFeign	Reçoit la source active (URL de l'API)	Savoir quelle API interroger	Dernières prévisions en base, sinon 503
MS5 → API météo	HTTPS	Envoie les coordonnées, reçoit 7 jours	Source des données	Prévisions déjà stockées
MS6 → MS5	Synchrone, OpenFeign	MS6 envoie villeId et date, reçoit la prévision du jour	Évaluation affichée immédiatement	MS6 gère l'erreur (404, 503)
MS5 → MS4	Asynchrone, RabbitMQ	Publie ForecastUpdated (villeId, dateGeneration)	Aucune réponse attendue, services découplés	Événement gardé dans la file, MS5 non bloqué

Diagramme de classe Prévisions





MS6 · Planificateur d'activités

Responsabilité

évaluer si une activité de plein air planifiée est réalisable selon les prévisions météo, et prévenir l'utilisateur lorsqu'elle ne l'est pas.

Base : PostgreSQL

les données de MS6 (type → activité → évaluation) sont liées entre elles PostgreSQL, base de données relationnelle, répond naturellement à ce besoin : il organise ces données en tables reliées et garantit leur cohérence.

Appelés : MS5 , MS3

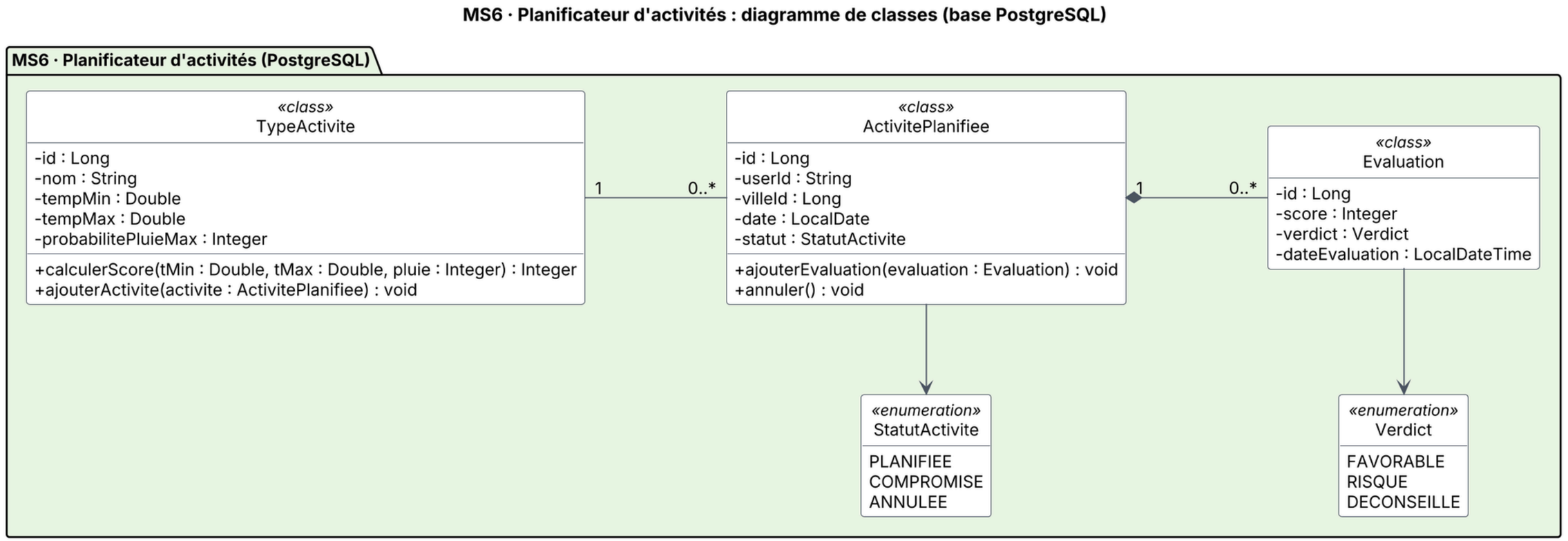
Appelant : Aucun microservice : seul l'utilisateur, via l'API Gateway

Evénement vers : MS4, via RabbitMQ

Lien	Mode	Données échangées	Pourquoi	Si indisponible
MS6 → MS5	Synchrone, OpenFeign	Envoie villeId et date, reçoit tMin, tMax et % de pluie	Score calculé immédiatement	503, rien n'est enregistré
MS6 → MS3	Synchrone, OpenFeign	Envoie villeId, reçoit la météo actuelle	Mesure réelle pour le jour même	Repli sur MS5, sinon 503
MS6 → MS4	Asynchrone, RabbitMQ	Publie ActivityRiskDetected (activityId, userId, villeId, date, verdict)	Aucune réponse attendue, services découplés	Événement gardé dans la file

Méthode	Endpoint	Rôle	Succès	Erreurs	Accès
GET	/api/types-activites	Lister les types d'activités	200	401	USER
POST	/api/types-activites	Créer un type d'activité	201	400, 401, 403	ADMIN
PUT	/api/types-activites/{id}	Modifier un type	200	400, 401, 403, 404	ADMIN
DELETE	/api/types-activites/{id}	Supprimer un type	204	401, 403, 404	ADMIN
POST	/api/activites	Planifier une activité	201	400, 401	USER
GET	/api/activites	Mes activités	200	401	USER
GET	/api/activites/{id}	Détail d'une activité	200	401, 404	USER
PUT	/api/activites/{id}	Modifier une activité (date, ville, type)	200	400, 401, 403, 404, 409	USER
PUT	/api/activites/{id}/annulation	Annuler une activité (statut ANNULEE)	200	401, 403, 404, 409	USER
POST	/api/activites/{id}/evaluations	Évaluer la faisabilité météo	201	401, 404, 503	USER
GET	/api/activites/{id}/evaluations	Historique des évaluations	200	401, 404	USER

Diagramme de classe Planificateur D’activites





MS7 · Signalements communautaires

Responsabilité

MS7 gère les signalements météo faits par la communauté : leur création, leur validation par les votes, et leur modération par l'administrateur.

Base : MySQL

MySQL, base relationnelle rapide et simple, convient à ces données légères et liées et très bien supporté par Django

Méthode	Endpoint	Rôle	Succès	Erreurs	Accès
GET	/api/signalements	Lister tous les signalements visibles (filtre type possible)	200	400	Public
GET	/api/signalements?villeId={villeId}	Signalements d'une ville	200	400	Public
GET	/api/signalements?statut=	Lister tous les signalements, dont EN_ATTENTE et REJETE, pour modérer	200	400, 401, 403	ADMIN
GET	/api/signalements/{id}	Détail d'un signalement et nombre de confirmations	200	404	Public
POST	/api/signalements	Signaler un phénomène météo	201	400, 401, 404, 503	USER
POST	/api/signalements/{id}/votes	Confirmer ou infirmer un signalement	201	400, 401, 404, 409	USER
POST	/api/signalements/{id}/moderations	Modérer : VALIDER ou SUPPRIMER	201	400, 401, 403, 404	ADMIN
GET	/api/signalements/{id}/moderations	Historique de modération	200	401, 403, 404	ADMIN

MS7 · Signalements communautaires

Appelés : **MS1, MS3**

Appelant : Aucun microservice : l'utilisateur et le visiteur, via l'API Gateway

Evénement vers : MS4, via RabbitMQ

Lien	Mode	Données échangées	Pourquoi	Si indisponible
MS7 → MS1	Synchrone, REST	Envoie villeId, reçoit la ville (existe ou non)	Vérifier la ville du signalement	503, signalement non créé
MS7 → MS3	Synchrone, REST	Envoie villeId, reçoit les mesures officielles	Comparer le signalement aux mesures officielles	Signalement enregistré sans comparaison
MS7 → MS4	Asynchrone, RabbitMQ	Publie NewWeatherReport (signalementId, villeId, type, statut)	Aucune réponse attendue, services découplés	Événement gardé dans la file, MS7 non bloqué

Diagramme de classe Signalements Communautaires

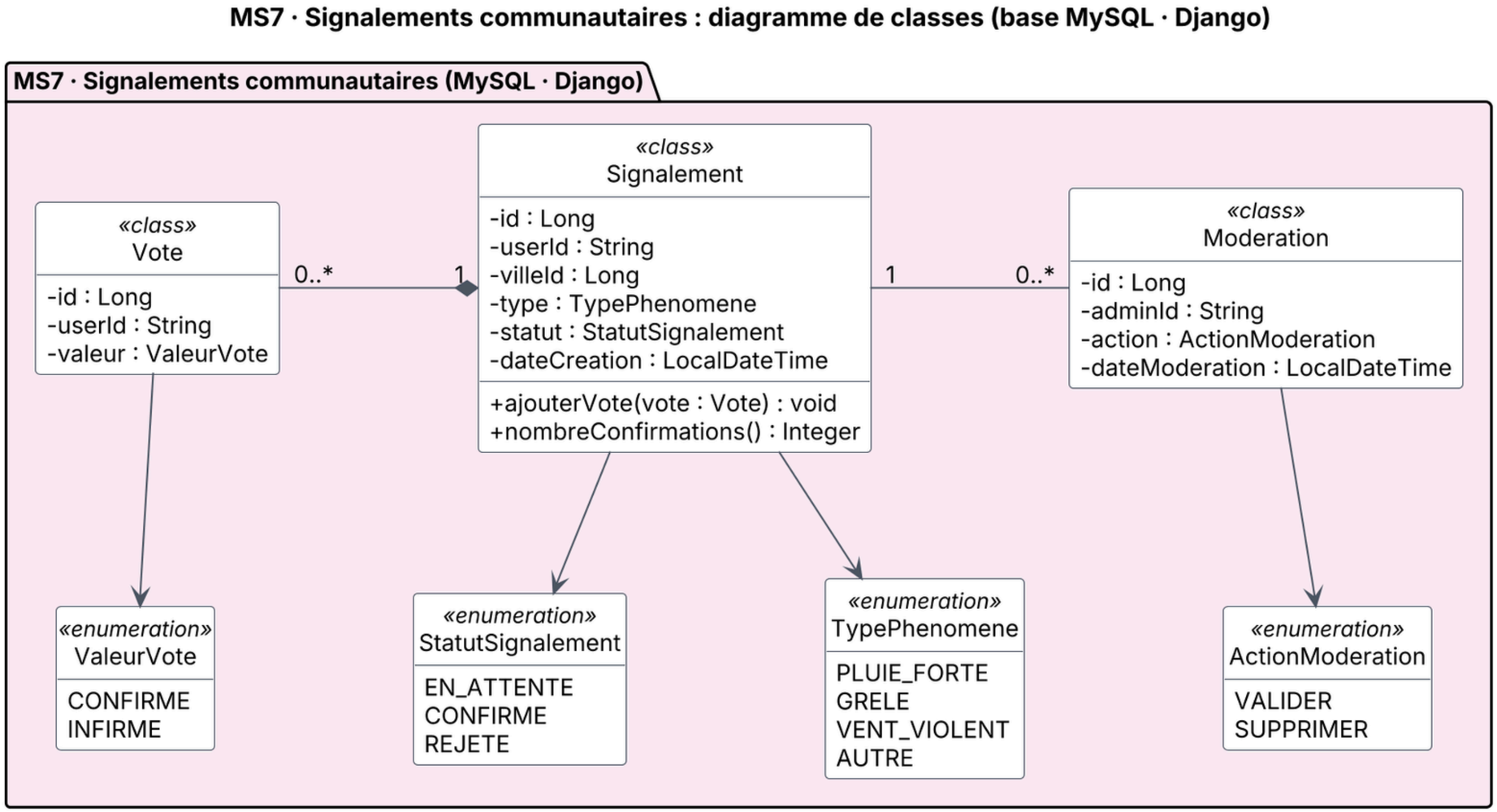
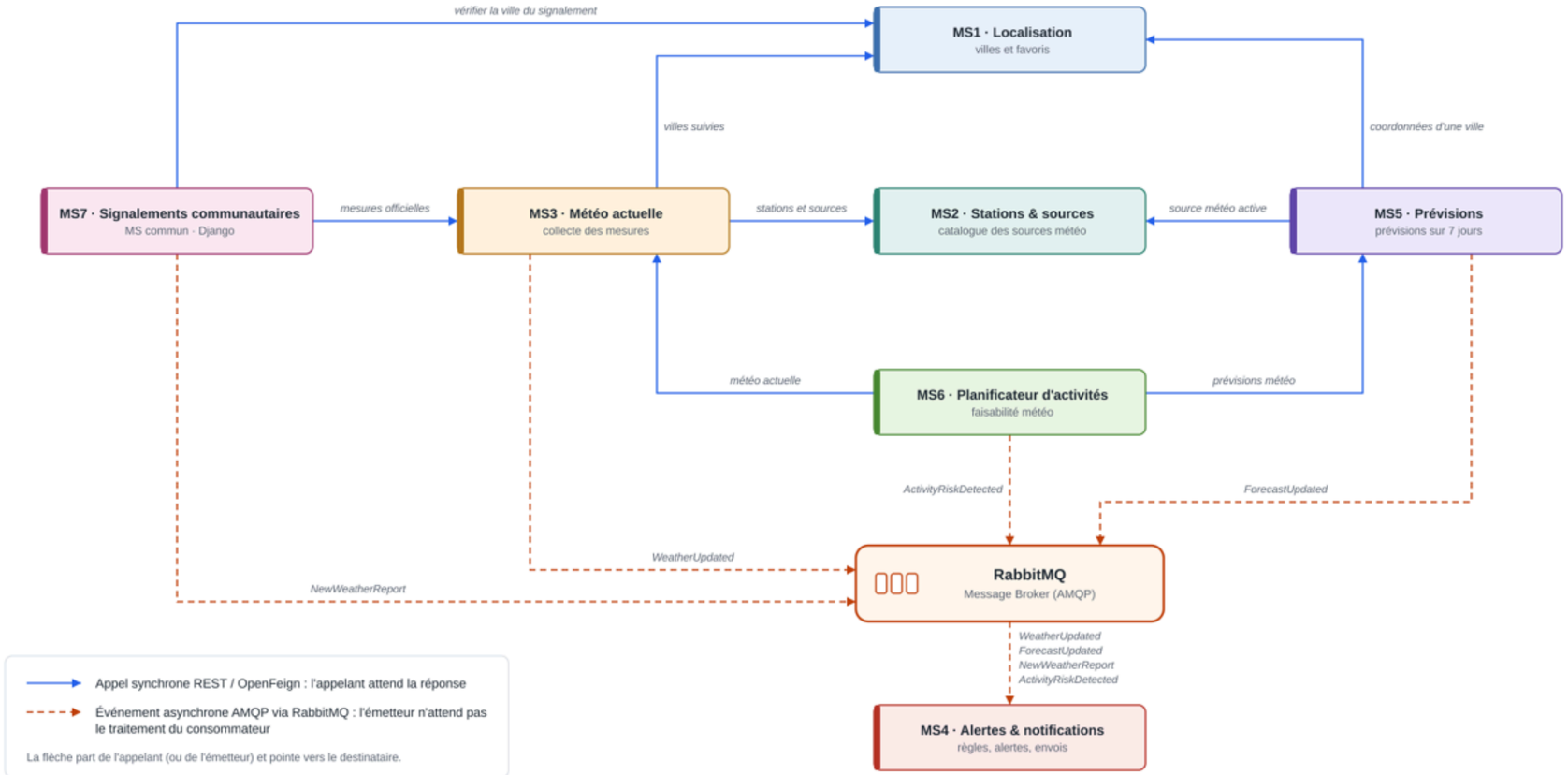


Diagramme de communication



Merci de votre attention

Questions ?